JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.temporal

Interface TemporalQuery<R>

Type Parameters:

`R - the type returned from the query`

Functional Interface:

`This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.`

```
@FunctionalInterface
public interface TemporalQuery<R>
```

Strategy for querying a temporal object.

Queries are a key tool for extracting information from temporal objects. They exist to externalize the process of querying, permitting different approaches, as per the strategy design pattern. Examples might be a query that checks if the date is the day before February 29th in a leap year, or calculates the number of days to your next birthday.

The `TemporalField` interface provides another mechanism for querying temporal objects. That interface is limited to returning a `long`. By contrast, queries can return any type.

There are two equivalent ways of using a `TemporalQuery`. The first is to invoke the method on this interface directly. The second is to use `TemporalAccessor.query(TemporalQuery)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisQuery.queryFrom(temporal);
temporal = temporal.query(thisQuery);
```

It is recommended to use the second approach, `query(TemporalQuery)`, as it is a lot clearer to read in code.

The most common implementations are method references, such as `LocalDate::from` and `ZoneId::from`. Additional common queries are provided as static methods in `TemporalQueries`.

Implementation Requirements:

`This interface places no restrictions on the mutability of implementations, however immutability is strongly recommended.`

Since:

`1.8`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
R	queryFrom(TemporalAccessor temporal)	Queries the specified temporal object.

Method Detail

queryFrom

`R queryFrom(TemporalAccessor temporal)`

Queries the specified temporal object.

This queries the specified temporal object to return an object using the logic encapsulated in the implementing class. Examples might be a query that checks if the date is the day before February 29th in a leap year, or calculates the number of days to your next birthday.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.query(TemporalQuery)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisQuery.queryFrom(temporal);
temporal = temporal.query(thisQuery);
```

It is recommended to use the second approach, `query(TemporalQuery)`, as it is a lot clearer to read in code.

Implementation Requirements:

The implementation must take the input object and query it. The implementation defines the logic of the query and is responsible for documenting that logic. It may use any method on TemporalAccessor to determine the result. The input object must not be altered.

The input temporal object may be in a calendar system other than ISO. Implementations may choose to document compatibility with other calendar systems, or reject non-ISO temporal objects by querying the chronology.

This method may be called from multiple threads in parallel. It must be thread-safe when invoked.

Parameters:

temporal - the temporal object to query, not null

Returns:

the queried value, may return null to indicate not found

Throws:

DateTimeException - if unable to query

ArithmeticException - if numeric overflow occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.